

Tree-Cotree Gauge Variants: Derivation

A-Phi Solver project

September 2026

Contents

1. Setup and notation	1
2. The “essential incidence matrix” F, from first principles	3
3. Why $\{\mathbf{a} : G^T \mathbf{a} = 0\}$ is a valid gauge choice	3
4. The reduced system	4
5. Why Method D is “unsymmetric” (and Method E would be “symmetric”)	5
6. What this derivation does and doesn’t rely on the paper for	6

Written in response to a direct question about where the Method D (Munteanu “unsymmetric”) reduced-matrix formula actually comes from. docs/REFERENCES.md records what was verified directly against I. Munteanu, “Tree-cotree condensation properties,” and discloses that her own equations (19)-(20), as extracted from the PDF, showed what looks like a symbol collision (“ F ” used for both “number of mesh faces” and “the essential incidence matrix” in different sections) — too risky to copy verbatim. This document is the derivation that was done instead, from her stated general framework and explicit basis choices, worked through fully and independently rather than summarized. It matches src/gauge_variants.cpp exactly; if the two ever disagree, the code’s behavior is the bug.

1. Setup and notation

Let $\mathbf{a} \in \mathbb{R}^E$ be the vector of edge-based (Whitney/Nédélec, first-order) degrees of freedom for the magnetic vector potential $\mathbf{A}$, one per mesh edge ($E = \text{mesh.num_edges}()$). The curl-curl system this project will eventually assemble in Phase 04 has the form

$$M\mathbf{a} = \mathbf{j}, \quad M = C^T \nu C \tag{1}$$

where C is the discrete curl matrix (`build_curl_matrix`, edges $\rightarrow$ faces) and ν is the (symmetric, positive semi-definite) material reluctivity matrix. M is symmetric and positive *semi*-definite, and singular: its null space is exactly the set of discrete gradient fields (Bossavit 1998, already cited in this project for the $CG = 0$ identity that Phase 02’s tests verify) — physically, adding $\nabla\varphi$ to $\mathbf{A}$ for any scalar φ doesn’t change $\mathbf{B} = \nabla \times \mathbf{A}$, so it doesn’t change what $M\mathbf{a}$ measures either. Gauging means adding enough independent linear constraints to remove exactly that null space, without removing any physically meaningful solution.

Phase 03 step 1 (`build_tree_cotree`) already does the topological half of this: it builds a spanning tree/forest over the mesh’s node graph (with each PEC body collapsed to a single reference/ground group, per `docs/ROADMAP.md` Phase 03 step 4), and partitions the E edges into **tree edges** (the ones used by that spanning tree/forest) and **cotree edges** (everything else). Relabel/reorder $\mathbf{a}$ ’s components accordingly:

$$\mathbf{a} = \begin{bmatrix} \mathbf{a}_c \\ \mathbf{a}_t \end{bmatrix}, \quad \mathbf{a}_c \in \mathbb{R}^{n_c}, \quad \mathbf{a}_t \in \mathbb{R}^{n_t} \quad (2)$$

where $n_c = \text{num_cotree}$ and $n_t = \text{num_tree}$. This is purely a re-indexing (no change of basis), matching `GaugeVariant::cotree_local_index` in the code.

Define the **reduced gradient matrix** G (size $E \times n_f$, with $n_f = \text{number of non-reference DOF groups} = n_t$ exactly, per the tree-edge-count invariant `tree_edge_count = num_groups - num_reference_groups` already checked in `tests/test_tree_cotree.cpp`): row $e = (i, j)$ (canonical $i < j$) has -1 at the column for `group(i)` if that group is free (0 if it’s a reference/PEC body, since its potential is fixed and not a free unknown), and $+1$ at the column for `group(j)` under the same rule. This is exactly `build_gradient_matrix`’s own convention (`incidence.hpp`), just with reference-body columns dropped. Partition its rows the same cotree/tree way:

$$G = \begin{bmatrix} G_c \\ G_t \end{bmatrix}, \quad G_c \in \mathbb{R}^{n_c \times n_f}, \quad G_t \in \mathbb{R}^{n_t \times n_f} \text{ (square, } n_t = n_f) \quad (3)$$

Claim: G_t is invertible. This is the standard fact that the (ground-collapsed) incidence matrix of a spanning tree is nonsingular: every free group is reached by *exactly one* tree path back to some reference root (that’s what a tree/forest means), so G_t ’s rows/columns can be ordered (by BFS discovery order — exactly `TreeCotreeResult::parent_group / discovering_edge`, already computed) into a lower-triangular structure with ± 1 on the diagonal after a suitable permutation. This is *why* `compute_essential_incidence_matrix` can solve $G_t \mathbf{x} = \mathbf{b}$ by a single top-down tree walk (`solve_tree_system` in `gauge_variants.cpp`) instead of a general matrix inversion — it isn’t an optimization on top of a generic solve, it’s a direct reading of this triangular structure.

2. The “essential incidence matrix” F , from first principles

Define

$$F = G_c G_t^{-1} \in \mathbb{R}^{n_c \times n_f} \quad (4)$$

— Munteanu’s own name and definition, Sec. III.A, independently reconstructed below.

Claim: the condition $G^T \mathbf{a} = 0$ is equivalent to $\mathbf{a}_t = -F^T \mathbf{a}_c$.

Write $G^T = [G_c^T \ G_t^T]$ (columns of G^T are edges, so this is the same cotree/tree split applied to columns instead of rows). Then

$$G^T \mathbf{a} = G_c^T \mathbf{a}_c + G_t^T \mathbf{a}_t \quad (5)$$

Setting this to zero and solving for $\mathbf{a}_t$ (using that G_t , hence G_t^T , is invertible):

$$G_t^T \mathbf{a}_t = -G_c^T \mathbf{a}_c \quad (6)$$

$$\mathbf{a}_t = -(G_t^T)^{-1} G_c^T \mathbf{a}_c = -(G_c G_t^{-1})^T \mathbf{a}_c = -F^T \mathbf{a}_c \quad (7)$$

This derives the paper’s stated relation (her eq. 14, “ $G_0^T \mathbf{a} = 0 \implies \mathbf{a}_t = -F^T \mathbf{a}_c$ ”, which was extracted cleanly and unambiguously, unlike eqs. 19–20) directly from the definition of F , rather than needing to trust that extraction at all.

3. Why $\{\mathbf{a} : G^T \mathbf{a} = 0\}$ is a valid gauge choice

Call $S_s = \{\mathbf{a} \in \mathbb{R}^E : G^T \mathbf{a} = 0\}$ (“solenoidal”, Munteanu’s term, Sec. V). Two things need checking for this to be a legitimate gauge-fixing: S_s must have the right dimension, and it must meet every physically-equivalent “coset” $\mathbf{a} + \text{Range}(G)$ (the set of vector potentials that differ only by a gradient, hence give the same $\mathbf{B}$ field) in *exactly one point*.

Dimension. G has full column rank n_f (immediate from G_t being an invertible $n_f \times n_f$ block of it), so G^T has rank n_f too, and $S_s = \ker(G^T)$ has dimension $E - n_f = n_c$.

The parametrization $\mathbf{a} = \begin{bmatrix} \mathbf{a}_c \\ -F^T \mathbf{a}_c \end{bmatrix}$ from Section 2, as $\mathbf{a}_c$ ranges over all of $\mathbb{R}^{n_c}$, traces out exactly this n_c -dimensional space.

Trivial intersection with $\text{Range}(G)$. Suppose $G\varphi \in S_s$ for some free-group potential vector φ , i.e. $G^T(G\varphi) = 0$. Dot both sides with φ :

$$0 = \varphi^T G^T G \varphi = (G\varphi) \cdot (G\varphi) = \|G\varphi\|^2 \implies G\varphi = 0 \quad (8)$$

So $S_s \cap \text{Range}(G) = \{0\}$. Since $\dim(S_s) + \dim(\text{Range}(G)) = n_c + n_f = E$, this gives

$$\mathbb{R}^E = S_s \oplus \text{Range}(G) \quad (9)$$

(a direct sum) — every coset $\mathbf{a} + \text{Range}(G)$ meets S_s in exactly one point. That point is what Method D solves for.

(The same argument, run on $\{\mathbf{a} : \mathbf{a}_t = 0\}$ instead of S_s , shows *why* Method A’s cruder gauge choice is equally valid: if $G\varphi$ has $(G\varphi)_t = 0$, i.e. $G_t\varphi = 0$, then G_t invertible forces $\varphi = 0$, hence $G\varphi = 0$ — the same trivial-intersection argument, just against a different n_c -dimensional subspace. Both methods are legitimate gauge choices for the same underlying reason: G_t ’s invertibility.)

4. The reduced system

Both variants share the same **test space**: Munteanu’s stated choice (Sec. V) is the plain cotree-row selector $W = \begin{bmatrix} I \\ 0 \end{bmatrix}$ (an $E \times n_c$ matrix whose columns are the standard basis vectors for the cotree positions) — i.e., “test” the full system $M\mathbf{a} = \mathbf{j}$ by keeping only its cotree rows, discarding the tree rows entirely. This is a genuine modeling choice attributable to her (not re-derivable from anything more fundamental) and is what both `build_albanese_rubinacci_gauge` and `build_munteanu_unsymmetric_gauge` do identically via `select_cotree_entries`.

Where the two methods differ is the **trial space** — how $\mathbf{a}_t$ is expressed in terms of $\mathbf{a}_c$ before substituting into those cotree rows.

Write M in cotree/tree block form (symmetric, so $M_{tc} = M_{ct}^T$):

$$M = \begin{bmatrix} M_{cc} & M_{ct} \\ M_{tc} & M_{tt} \end{bmatrix} \quad (10)$$

Method A (Albanese–Rubinacci): trial space is literally $\mathbf{a}_t = 0$. Substituting into the cotree rows of $M\mathbf{a} = \mathbf{j}$:

$$(M\mathbf{a})_{\text{cotree}} = M_{cc}\mathbf{a}_c + M_{ct} \cdot 0 = M_{cc}\mathbf{a}_c = \mathbf{j}_c \quad (11)$$

so the reduced system is simply

$$M_{cc}\mathbf{a}_c = \mathbf{j}_c \quad (12)$$

— exactly M ’s own cotree-cotree principal submatrix, which is why it preserves M ’s sparsity exactly (it’s a genuine submatrix, not a product of anything).

Method D (Munteanu unsymmetric): trial space is $\mathbf{a} = L^T \mathbf{a}_c$ where $L = [I \ -F]$, i.e. $\mathbf{a}_t = -F^T \mathbf{a}_c$ (Section 2’s solenoidal condition). Substituting into the cotree rows of $M\mathbf{a} = \mathbf{j}$:

$$(M\mathbf{a})_{\text{cotree}} = M_{cc}\mathbf{a}_c + M_{ct}\mathbf{a}_t = M_{cc}\mathbf{a}_c + M_{ct}(-F^T \mathbf{a}_c) = (M_{cc} - M_{ct}F^T) \mathbf{a}_c = \mathbf{j}_c \quad (13)$$

so the reduced system is

$$(M_{cc} - M_{ct}F^T) \mathbf{a}_c = \mathbf{j}_c \quad (14)$$

— this is exactly `build_munteanu_unsymmetric_gauge`’s product matrix (computed there as `(select cotree rows of M) * L^T`, the same expression written out in dense matrix form rather than block-algebra form).

5. Why Method D is “unsymmetric” (and Method E would be “symmetric”)

M_{cc} is symmetric (a principal submatrix of a symmetric matrix always is). But $M_{ct}F^T$ has no reason to be:

$$(M_{ct}F^T)^T = FM_{ct}^T = FM_{tc} \quad (15)$$

and there’s nothing forcing $FM_{tc} = M_{ct}F^T$ in general, since F comes purely from the mesh’s tree topology (Section 2) and has no relationship to M ’s specific entries. So $M_{cc} - M_{ct}F^T$ is generically **not** symmetric, even though the original M is — exactly why Munteanu calls this variant “unsymmetric”, and a useful independent check that this derivation lines up with her own naming rather than having gone wrong somewhere.

For contrast (not implemented in this project, since it isn’t one of the two variants asked for, but worth noting for why the name split makes sense): Munteanu’s “symmetric” variant (her Method E) instead uses $W = V = L^T$ for *both* test and trial spaces, giving a reduced matrix $LM L^T$. That form is automatically symmetric for *any* L , since

$$(LM L^T)^T = LM^T L^T = LM L^T \quad (16)$$

whenever $M = M^T$ — symmetric by construction, regardless of what L is, which is exactly the distinguishing feature the name refers to.

6. What this derivation does and doesn't rely on the paper for

Attributed to Munteanu's paper, and taken as a design choice rather than re-derived: the specific *names* Albanese-Rubinacci/unsymmetric for these two constructions, the choice of test space $W = \begin{bmatrix} I \\ 0 \end{bmatrix}$ for both, and the reported condition-number ordering used only as a direction-of-comparison sanity check (not as ground truth for these tiny test meshes).

Derived independently in this document, and not dependent on trusting any specific equation's OCR/extraction fidelity: G_t 's invertibility, the equivalence of $G^T \mathbf{a} = 0$ and $\mathbf{a}_t = -F^T \mathbf{a}_c$, the transversality argument for both gauge choices, the block-algebra reduction to $M_{cc} \mathbf{a}_c = \mathbf{j}_c$ (Method A) and $(M_{cc} - M_{ct} F^T) \mathbf{a}_c = \mathbf{j}_c$ (Method D), and the symmetric-vs-unsymmetric contrast in Section 5.

`tests/test_gauge_variants.cpp` checks the *result* of this derivation two independent ways: Method A's reduced matrix is checked directly against M 's dense cotree submatrix, and both methods are checked to recover a vector potential whose curl matches $\nabla \times \mathbf{z}$ for a manufactured, self-consistent source $\mathbf{j} = M\mathbf{z}$ — a check that would fail if the block algebra above had a sign or transpose error, regardless of whether the general approach was sound.